

UANL



FCFM

Universidad Autónoma de Nuevo León

Facultad de Ciencias Físico Matemáticas

Algoritmia y Optimización

Algorithm Optimization

Reporte de tarea — Segundo parcial

Docente: M.C. José Anastacio Hernández Saldaña

Alumno: Sebastián Hernández Pérez

Matrícula: 2162902

Grupo: 032

Repositorio: `https://github.com/Sebastian-hdz/AYO_ejercicios_segundo_parcial/tree/main`

Video demo: `https://canva.link/3l3ocztqghu32j4`

25 de octubre de 2026

Introducción y entorno experimental

Este reporte presenta la implementación y el análisis de cinco ejercicios sobre algoritmos de optimización: aceleración paralela, el algoritmo voraz de la mochila, búsqueda local 2-opt para el problema del agente viajero (TSP), recocido simulado y la aproximación al TSP basada en árbol de expansión mínimo (MST). Todo el código está en el repositorio indicado en la portada y las gráficas se generan con `matplotlib`.

Los experimentos se ejecutaron en Windows con Python 3.12 en un equipo con 4 núcleos lógicos. Los experimentos aleatorios usan semillas fijas, por lo que los resultados de los ejercicios 2 a 5 son reproducibles (los tiempos de ejecución dependen del equipo). Para tener un valor de referencia $C_{\text{óptimo}}$ en los ejercicios 3 y 5 se calculó el tour óptimo con el algoritmo de programación dinámica de Held–Karp, de complejidad $O(n^2 2^n)$.

1. Ejercicio 1: Aceleración paralela

A. Metodología

Se utilizó la función `estimate_pi` vista en clase (estimación Monte Carlo de π con 1,000,000 de muestras por corrida) sobre 8 corridas independientes, una por semilla (0 a 7). Cada configuración se ejecutó con `multiprocessing.Pool` con 1, 2, 4 y 8 procesos; `pool.map` reparte las semillas entre los procesos y recolecta los resultados.

El speedup se define como $S(p) = T_{\text{seq}}/T_p$, donde T_{seq} es el tiempo del ciclo secuencial (sin pool) y T_p es el tiempo con p procesos, incluyendo la creación del pool. La eficiencia es $S(p)/p$. Cada medición se repitió dos veces y se reportó el menor tiempo. Como complemento se repitió el experimento con `ThreadPoolExecutor` (hilos). Para la ley de Amdahl, $S(p) = 1/\big(f + (1-f)/p\big)$, se estimó la fracción no paralelizable f con el speedup medido con 8 procesos.

B. Resultados y visualizaciones

El tiempo del ciclo secuencial fue de $T_{\text{seq}} = 3{,}836$ s.

Procesos / hilos	Tiempo procesos (s)	Speedup	Eficiencia	Tiempo hilos (s)	Speedup
1	3.800	1.01	101 %	2.700	1.42
2	3.265	1.17	59 %	2.656	1.44
4	3.858	0.99	25 %	2.642	1.45
8	5.856	0.65	8 %	2.664	1.44

Cuadro 1: Tiempos, speedup y eficiencia de `estimate_pi` (8 corridas $\times$ 10^6 muestras).

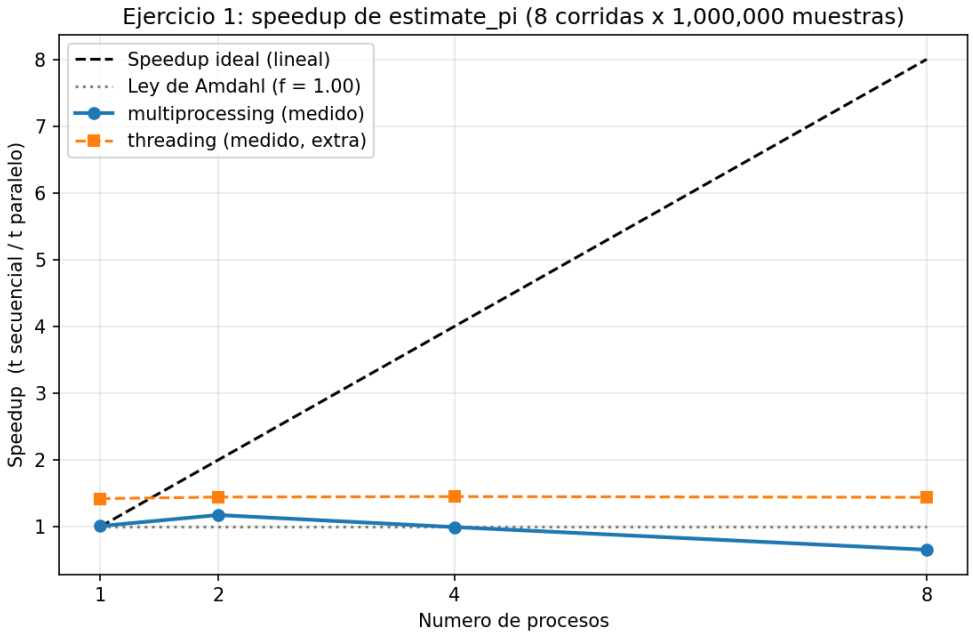


Figura 1: Speedup contra número de procesos. La línea punteada negra es el speedup ideal; la gris, el modelo de Amdahl con la fracción f estimada; la azul, `multiprocessing`; la naranja, `threading` (extra).

C. Análisis de resultados

¿Por qué la aceleración no es perfectamente lineal? En este experimento la aceleración con procesos es casi inexistente (máximo 1,17× con 2 procesos) y con 8 procesos es incluso menor que 1. Los factores que lo explican son:

1. **Sobrecostos de arranque y comunicación.** En Windows, `multiprocessing` crea los procesos con el método *spawn*: cada proceso inicia un intérprete nuevo, vuelve a importar el script principal (en mi implementación esto incluye la biblioteca `matplotlib`) y recibe las semillas y devuelve los resultados mediante serialización. Este costo está incluido en T_p y crece con el número de procesos. Cada tarea dura solo $\approx 0{,}48$ s (3,836/8), así que el costo de arranque es comparable al trabajo útil. Por eso con 8 procesos el tiempo (5.856 s) supera al secuencial.

2. **Ley de Amdahl.** Existe una fracción f del programa que no se paraleliza (crear el pool, repartir las semillas, recolectar y promediar), que limita el speedup a $1/f$ sin importar cuántos procesos se usen. Con $p = 2$ la fórmula $f = \big(1/S - 1/p\big)/\big(1 - 1/p\big)$ da $f \approx 0{,}70$. El valor $f = 1{,}00$ de la gráfica es un artefacto del ajuste: como con 8 procesos $S < 1$, la fórmula da $f > 1$ y se recorta a 1. Además, Amdahl por sí sola no basta, porque predice que S nunca disminuye al aumentar p, y aquí baja de 1,17 a 0,65; es decir, el sobrecosto crece con p.

3. **Sincronización y contención de recursos.** Con 8 procesos sobre 4 núcleos lógicos hay más procesos que núcleos, lo que produce cambios de contexto. Además, los núcleos lógicos pueden compartir unidades de ejecución de un mismo núcleo físico, por lo que 4 núcleos lógicos no equivalen a 4 veces el rendimiento de uno.

4. **Granularidad y balanceo de carga.** Solo hay 8 tareas; con $p = 2$ o $p = 4$ el tiempo total lo determina el proceso más cargado, y con $p = 8$ cada proceso ejecuta una sola tarea y el arranque domina.

Hilos (`threading`). El speedup con hilos es aproximadamente 1,4× y constante desde 1 hilo. Como con un solo hilo ya es 1,42×, ese desfase no proviene del paralelismo, sino probablemente de variaciones en la medición de la línea base (estado del procesador, procesos en segundo plano). Respecto al tiempo con un hilo, el speedup con 2, 4 y 8 hilos es 1,02, 1,02 y 1,01: no hay aceleración. Esto se debe al *Global Interpreter Lock* (GIL) de CPython, que impide que dos hilos ejecuten código Python (tarea limitada por CPU) al mismo tiempo.

Conclusión del ejercicio. Con tareas de aproximadamente medio segundo, el costo de crear y comunicar procesos en Windows domina sobre el cómputo. Para acercarse al speedup ideal habría que usar tareas más grandes, crear el pool una sola vez y reutilizarlo, evitar importaciones pesadas en los procesos hijos y no usar más procesos que núcleos.

2. Ejercicio 2: Análisis algorítmico (Greedy Knapsack)

A. Metodología

Se implementó un algoritmo voraz genérico que ordena los objetos con un criterio y los agrega mientras quepan en la mochila. Se comparan dos criterios: (i) **menor peso primero** (orden ascendente por w_i) y (ii) **valor/peso** (orden descendente por v_i/w_i). Como referencia se calculó el óptimo con programación dinámica. Se evaluaron los objetos del PDF de clase y una instancia diseñada: capacidad $W = 10$ y cuatro objetos $A(v = 1, w = 1)$, $B(v = 1, w = 2)$, $C(v = 1, w = 3)$ y $D(v = 50, w = 5)$.

B. Resultados y visualizaciones

Instancia	Método	Objetos elegidos	Capacidad usada	Valor total
PDF ($W = 50$)	Menor peso primero	A, B	30/50	160
	Valor/peso	A, B	30/50	160
	Óptimo (DP)	B, C	50/50	220
Diseñada ($W = 10$)	Menor peso primero	A, B, C	6/10	3
	Valor/peso	A, B, D	8/10	52
	Óptimo (DP)	—	—	52

Cuadro 2: Comparación de los dos criterios voraces.

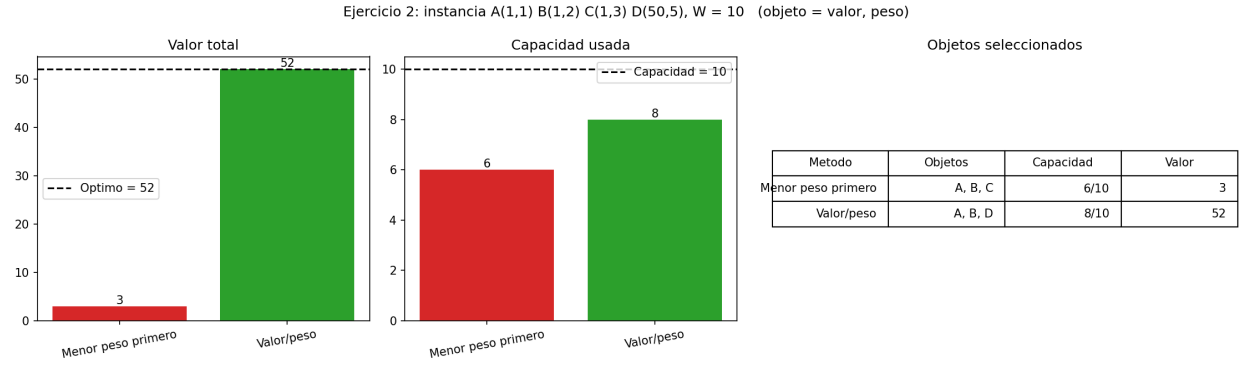


Figura 2: Instancia diseñada ($W = 10$): valor total, capacidad usada y objetos seleccionados por cada método.

C. Análisis de resultados

Instancia donde menor peso es peor que valor/peso. En la instancia diseñada, el criterio de menor peso toma A, B y C (peso 6, valor 3) y ya no cabe D, porque $6+5=11>10$. El criterio valor/peso ordena por razón ($D=10$, $A=1$, $B=0{,}5$, $C=0{,}33$) y toma D, A y B (peso 8, valor 52). El resultado de menor peso (3) es mucho peor que el de valor/peso (52), que además coincide con el óptimo de programación dinámica.

Explicación. El criterio de menor peso ignora el valor: llena la mochila con objetos ligeros pero casi sin valor y deja sin espacio al único objeto que realmente importa. El criterio valor/peso mide cuánto valor aporta cada unidad de capacidad. Con los objetos del PDF ambos criterios empatan (160) y ambos quedan por debajo del óptimo (220), lo que muestra que ni siquiera valor/peso garantiza el óptimo en la mochila 0/1; en nuestra instancia coincide con él.

3. Ejercicio 3: Búsqueda local (2-opt para TSP)

A. Metodología

Se implementó 2-opt como búsqueda de ascenso de colina: para cada par de posiciones (i, k) se invierte el segmento $[i, k]$ del tour y, si el nuevo tour es más corto, se acepta; el proceso se repite hasta que ninguna inversión mejora el tour (óptimo local). Cada movimiento aceptado cuenta como una *mejora*.

El experimento usa 200 instancias aleatorias de 10 ciudades (coordenadas uniformes en $[0, 100]^2$, semilla fija). Para cada instancia se parte de (1) un tour aleatorio y (2) un tour de Vecino Más Cercano iniciando en una ciudad aleatoria. Se registra el número de mejoras y el *gap* del óptimo local respecto al óptimo, $\text{gap} = (C_{\text{local}}/C_{\text{óptimo}} - 1) \times 100\,\%$.

B. Resultados y visualizaciones

Métrica	Inicio aleatorio	Inicio NN
Costo inicial / óptimo	1.84	1.11
Mejoras (media)	9.71	2.65
Mejoras (mediana)	9	2
Mejoras (máximo)	19	11
Gap del óptimo local (media)	0.91 %	0.92 %
Instancias donde llega al óptimo	143/200	150/200

Cuadro 3: Resultados de 2-opt en 200 instancias de 10 ciudades.

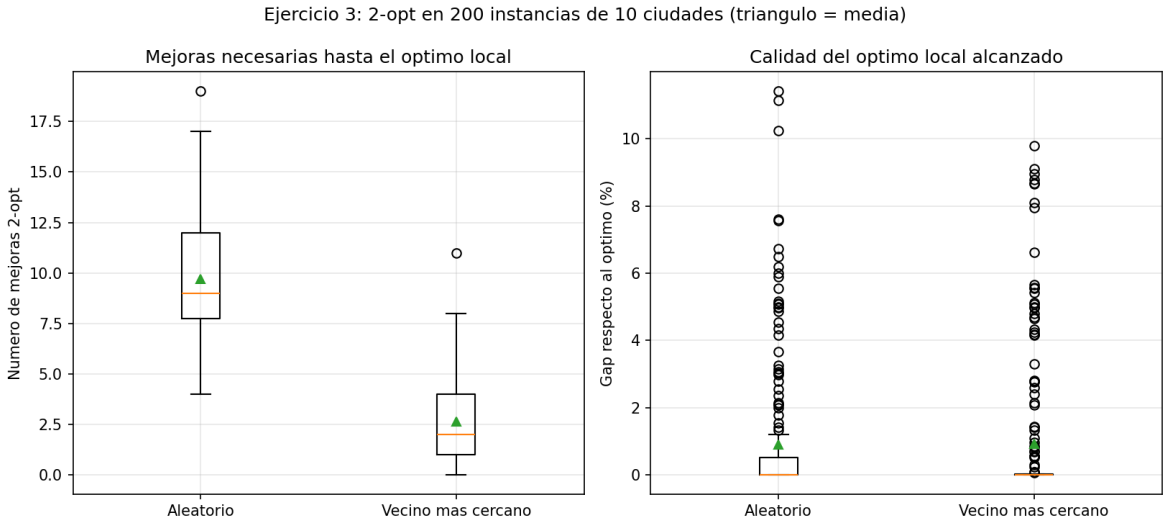


Figura 3: Izquierda: número de mejoras 2-opt hasta el óptimo local. Derecha: gap respecto al óptimo. El triángulo verde es la media.

C. Análisis de resultados

¿El inicio con Vecino Más Cercano requiere menos mejoras? Sí. En el diagrama de caja de la izquierda, la mediana baja de 9 mejoras (inicio aleatorio) a 2 (inicio NN); la media baja de 9.71 a 2.65 y el máximo de 19 a 11. Las cajas intercuartílicas casi no se traslapan (aproximadamente 8 a 12 mejoras contra 1 a 4). Además, comparando instancia por instancia, NN necesitó menos mejoras en 194 de las 200 instancias (igual en 3 y más en 3).

La razón es que NN ya construye un tour razonable: su costo inicial es 1,11 veces el óptimo, contra 1,84 veces de un tour aleatorio, que tiene muchos cruces que 2-opt debe eliminar uno por uno.

Calidad del óptimo alcanzado. La gráfica de la derecha muestra que la calidad final es prácticamente la misma: la mediana del gap es 0 % en ambos casos y la media es 0.91 % contra 0.92 %. Ambos métodos tienen valores atípicos de hasta aproximadamente 10 % y llegan al óptimo global en 143 y 150 de 200 instancias; esta diferencia es pequeña y no concluyente. Es decir, NN ahorra trabajo, pero no mejora el óptimo local, porque 2-opt es una búsqueda de ascenso de colina y

puede quedarse atrapada en óptimos locales desde cualquier inicio. Este es el problema que motiva el recocido simulado.

4. Ejercicio 4: Ajuste de parámetros (Simulated Annealing)

A. Metodología

Se implementó el recocido simulado del pseudocódigo visto en clase, para minimización: se parte de un tour aleatorio, el vecino se genera con una inversión 2-opt al azar, se define $\Delta E = L(\text{actual}) - L(\text{siguiente})$ y se acepta el movimiento si $\Delta E > 0$ o, en caso contrario, con probabilidad $e^{\Delta E/T}$. El algoritmo termina cuando el esquema de enfriamiento devuelve $T = 0$.

Se usó una instancia de 30 ciudades (la ruta inicial aleatoria tiene costo 1591,9) y tres esquemas de enfriamiento, todos de $T_0 = 100$ a $T_{\text{mín}} = 0{,}1$ en N iteraciones:

$$\begin{aligned} \text{Lineal:} \quad & T(t) = T_0 - (T_0 - T_{\text{mín}})\, t/N \\ \text{Exponencial:} \quad & T(t) = T_0\, \alpha^t, \quad \alpha = (T_{\text{mín}}/T_0)^{1/N} \\ \text{Logarítmico:} \quad & T(t) = \frac{T_0}{1 + c\ln(1+t)}, \quad c = \frac{T_0/T_{\text{mín}} - 1}{\ln(1+N)} \end{aligned}$$

Las curvas de convergencia usan $N = 20{,}000$ y el promedio de 8 corridas. Para comparar tiempo y calidad se varió $N \in \{1{,}000;\ 3{,}000;\ 10{,}000;\ 30{,}000\}$ con 8 corridas por configuración, midiendo el tiempo promedio y el costo final promedio. Como no se conoce el óptimo de 30 ciudades, el gap se calcula respecto al mejor costo promedio encontrado (487.1).

B. Resultados y visualizaciones

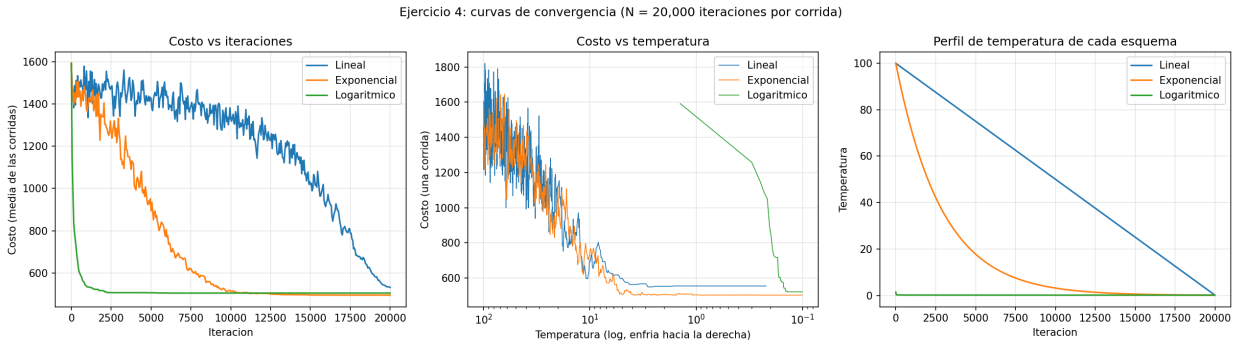


Figura 4: Curvas de convergencia: costo contra iteraciones (media de 8 corridas), costo contra temperatura (una corrida) y perfil de temperatura de cada esquema.

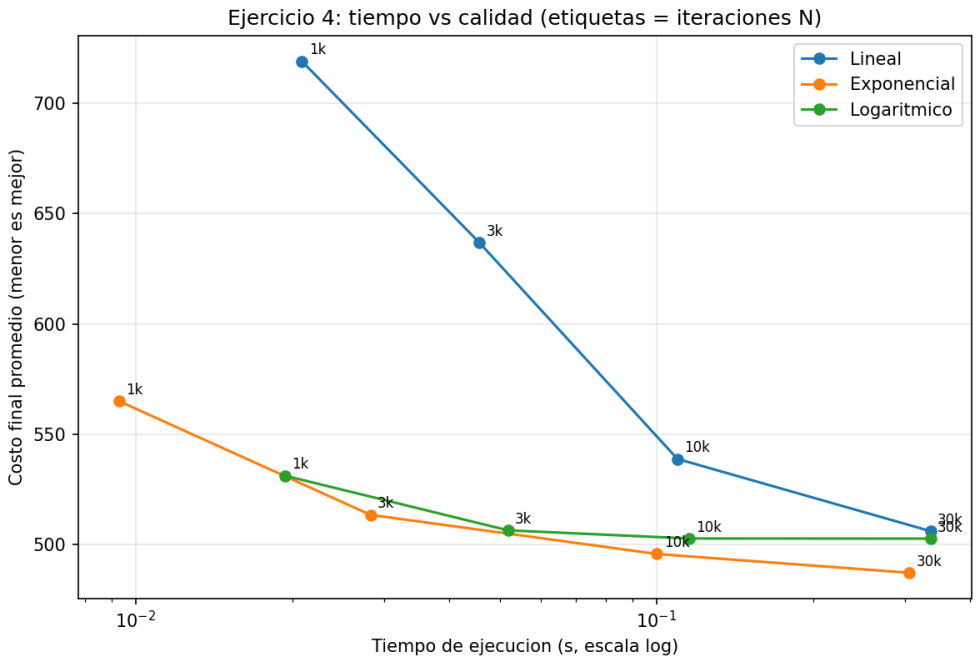


Figura 5: Tiempo de ejecución contra costo final promedio. Las etiquetas indican el número de iteraciones N.

Esquema	N	Costo final	Gap (%)	Tiempo (s)
Lineal	1,000	719.1	47.6	0.021
	3,000	636.8	30.7	0.046
	10,000	538.7	10.6	0.110
	30,000	506.0	3.9	0.336
Exponencial	1,000	565.0	16.0	0.009
	3,000	513.4	5.4	0.028
	10,000	495.7	1.7	0.100
	30,000	487.1	0.0	0.306
Logarítmico	1,000	531.1	9.0	0.019
	3,000	506.4	4.0	0.052
	10,000	502.7	3.2	0.115
	30,000	502.6	3.2	0.336

Cuadro 4: Costo final y tiempo promedio (8 corridas) por esquema y número de iteraciones.

C. Análisis de resultados

Impacto en la calidad.

- **Lineal.** La temperatura permanece alta durante la mayor parte de la corrida, por lo que se aceptan casi todos los movimientos y la búsqueda se comporta como una caminata aleatoria; el costo solo baja en el último tramo, cuando T es pequeña (panel izquierdo de la Figura 4).

Con presupuestos pequeños es el peor esquema (719.1 con $N = 1{,}000$) y solo se acerca a los demás con $N = 30{,}000$ (506.0).

- **Exponencial.** Reparte las iteraciones entre distintas escalas de temperatura: explora a T alta y refina a T baja. Converge de forma gradual y obtiene el mejor resultado con presupuestos grandes (495.7 con $N = 10{,}000$ y 487.1 con $N = 30{,}000$).

- **Logarítmico.** Con esta parametrización T cae casi de inmediato (por ejemplo, para $N = 20{,}000$, $T(1) \approx 1{,}4$), por lo que se comporta como un enfriamiento muy rápido, parecido a ascenso de colina con pequeñas perturbaciones. Converge en pocas iteraciones (la curva verde se aplana antes de las 2,500 iteraciones) y por eso es el mejor con presupuestos pequeños (531.1 con $N = 1{,}000$ y 506.4 con $N = 3{,}000$), pero se estanca en $\approx 502{,}6$ (3.2 % sobre el mejor) y ya no mejora al aumentar N.

Entre $N = 3{,}000$ y $N = 10{,}000$ el exponencial supera al logarítmico, y el lineal queda dominado por el exponencial en todo el rango. En la gráfica de costo contra temperatura se observa que, por debajo de $T \approx 10$ (exponencial) y $T \approx 3$ (lineal), las curvas se aplanan: la búsqueda ya es prácticamente ascenso de colina.

Impacto en el tiempo de cómputo. El tiempo depende del número de iteraciones y no del esquema, porque cada iteración cuesta lo mismo (una inversión 2-opt y el cálculo de la longitud del tour): con $N = 30{,}000$ los tres esquemas tardan entre 0.31 y 0.34 s, es decir, unos 10 μs por iteración. Las diferencias con $N = 1{,}000$ (0.009 s contra 0.021 s) son del orden de milisegundos y están dentro del ruido de medición. Más tiempo compra mejor calidad con rendimientos decrecientes: el exponencial pasa de 565.0 a 487.1 al multiplicar el tiempo por unas 34 veces, y la mayor parte de la mejora ocurre en los primeros incrementos de N.

Recomendación. Con presupuestos muy pequeños (hasta unos 3,000 iteraciones) conviene el enfriamiento rápido (logarítmico en esta parametrización); con presupuestos mayores, el exponencial. El lineal no es recomendable. Estos resultados provienen de una sola instancia de 30 ciudades, por lo que la generalización está limitada.

5. Ejercicio 5: Razón de aproximación (MST)

A. Metodología

Se generaron 100 instancias aleatorias de 9 ciudades (coordenadas uniformes en $[0, 10]^2$). Para cada una se construyó el MST con el algoritmo de Prim, se recorrió en preorden saltando las ciudades repetidas (atajos) y se obtuvo el tour de la aproximación; el óptimo se calculó con Held–Karp. La razón de aproximación es $C_{\text{obtenido}}/C_{\text{óptimo}}$.

Se compararon cuatro métodos, todos normalizados por el óptimo de su instancia: (1) **MST puro**; (2) **MST + 2-opt**, que aplica la búsqueda local 2-opt al tour del MST; (3) **búsqueda local**, 2-opt desde un tour aleatorio (Ejercicio 3); y (4) **recocido simulado** (Ejercicio 4) desde un tour aleatorio, con esquema exponencial, $T_0 = 5$, $T_{\text{mín}} = 0{,}01$ y $N = 3{,}000$ iteraciones.

B. Resultados y visualizaciones

Método	Media	Mediana	Máxima	Óptimo exacto
MST puro	1.145	1.148	1.379	5/100
MST + 2-opt	1.006	1.000	1.084	78/100
Búsqueda local (2-opt)	1.007	1.000	1.094	72/100
Recocido simulado	1.001	1.000	1.039	92/100

Cuadro 5: Razón $C_{\text{obtenido}}/C_{\text{óptimo}}$ en 100 instancias de 9 ciudades.

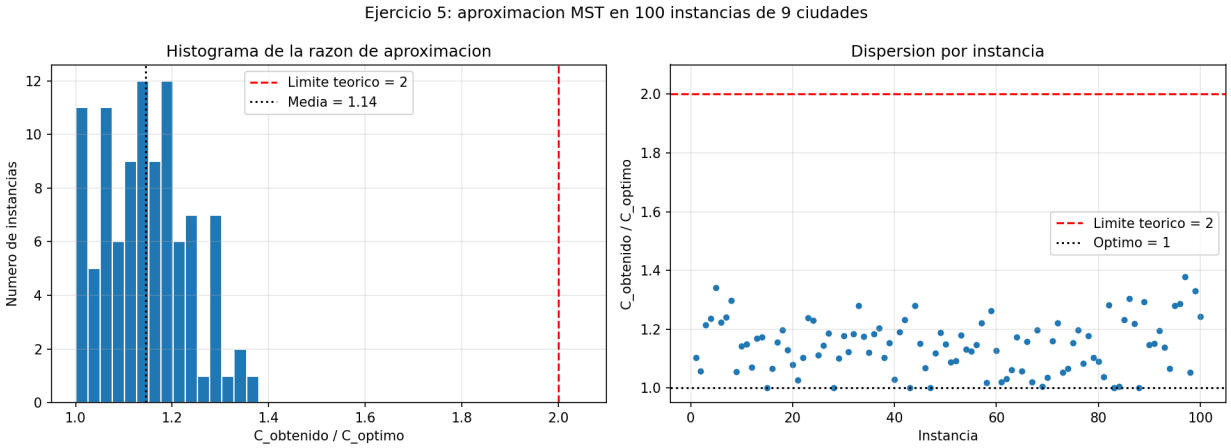


Figura 6: Razón de aproximación del MST: histograma (izquierda) y dispersión por instancia (derecha). La línea roja marca el límite teórico de 2 y la línea punteada negra el óptimo (1).

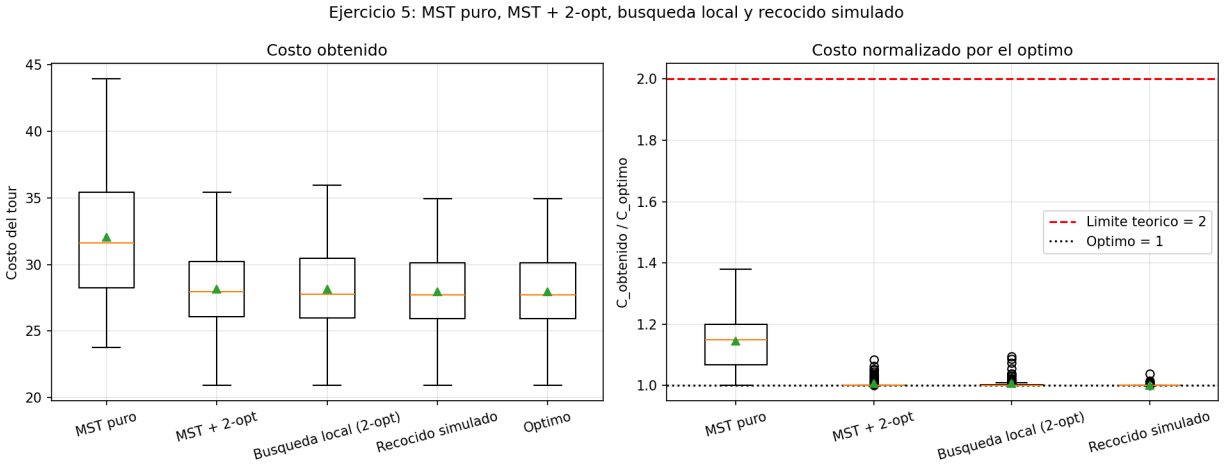


Figura 7: Comparación de MST puro, MST + 2-opt, búsqueda local y recocido simulado: costo del tour (izquierda) y costo normalizado por el óptimo (derecha).

C. Análisis de resultados

¿Qué tan cerca se mantiene la razón observada del límite teórico de 2? Muy lejos. La razón promedio es 1.145, la mediana 1.148 y la máxima 1.379; ninguna de las 100 instancias supera 2. La peor instancia usa apenas el 38 % del margen entre 1 y 2, pues $(1{,}379-1)/(2-1) = 0{,}379$. El histograma se concentra entre 1.0 y 1.4 y en la dispersión todos los puntos quedan muy por debajo de la línea roja. El límite de 2 es una garantía de peor caso: se apoya en $w(T) \leq C_{\text{óptimo}}$ y en que el recorrido duplica cada arista del MST ($2\,w(T)$), y en instancias aleatorias estas cotas rara vez son ajustadas al mismo tiempo, además de que los atajos acortan el recorrido.

¿Qué impacto tiene aplicar 2-opt como post-procesamiento al MST? Es muy grande: la razón media baja de 1.145 a 1.006, lo que elimina el 96 % del exceso sobre el óptimo, el máximo baja de 1.379 a 1.084 y las instancias resueltas de forma óptima suben de 5 a 78 de 100. El tour del MST suele tener cruces que 2-opt elimina con pocas mejoras.

Comparación con los experimentos anteriores. MST + 2-opt es similar a la búsqueda local desde un tour aleatorio (media 1.007, máximo 1.094 y 72 óptimos), con una ligera ventaja en óptimos exactos y en el peor caso, aunque con 100 instancias esa diferencia es pequeña. Esto es coherente con el Ejercicio 3: partir de una solución constructiva buena reduce el trabajo de la búsqueda local, pero no cambia mucho el óptimo local alcanzado. El recocido simulado fue el más preciso (media 1.001, máximo 1.039 y 92 óptimos), a cambio de más cómputo (3,000 iteraciones por instancia), pues acepta movimientos peores para escapar de óptimos locales. Una ventaja de MST + 2-opt es que conserva la garantía de la aproximación: 2-opt solo acepta mejoras, así que el resultado nunca es peor que el tour del MST y, por tanto, sigue siendo menor o igual a $2\,C_{\text{óptimo}}$; la búsqueda local desde un tour aleatorio y el recocido simulado no tienen esa garantía.

6. Conclusiones generales

- **Paralelismo.** Paralelizar no acelera automáticamente: el speedup está limitado por los sobrecostos de crear y comunicar procesos, por la fracción secuencial (ley de Amdahl), por el número de núcleos y, en Python, por el GIL en el caso de los hilos. Con tareas cortas en Windows, el sobrecosto domina (speedup máximo de 1,17× y de 0,65× con 8 procesos).
- **Algoritmos voraces.** Un criterio voraz es tan bueno como la información que usa: menor peso primero ignora el valor y falla de forma drástica (3 contra 52); valor/peso es mejor, pero tampoco garantiza el óptimo en la mochila 0/1.
- **Búsqueda local.** 2-opt mejora rápidamente un tour, y más rápido si se parte de Vecino Más Cercano (media de 2.65 mejoras contra 9.71), pero termina en óptimos locales con una calidad similar (gap medio de $\approx 0{,}9\,\%$) sin importar el inicio.
- **Recocido simulado.** Aceptar movimientos peores permite escapar de óptimos locales, pero su desempeño depende del esquema de enfriamiento y del presupuesto de iteraciones: el enfriamiento rápido conviene con poco tiempo y el exponencial con más tiempo. El tiempo depende del número de iteraciones y no del esquema.
- **Aproximación.** El algoritmo basado en MST garantiza $C \leq 2\,C_{\text{óptimo}}$ y en la práctica se mantuvo lejos de esa cota (media de 1.145 y máxima de 1.379). Combinado con 2-opt mantiene la garantía y se acerca mucho al óptimo (media de 1.006).
- **Idea general.** No existe un método que gane en todos los criterios: hay un compromiso entre calidad de la solución, tiempo de cómputo y garantías teóricas, como en el frente de Pareto

visto en clase. Los resultados de los ejercicios 3 a 5 provienen de instancias pequeñas (9 a 30 ciudades), con las que se puede calcular el óptimo exacto, por lo que no deben extrapolarse sin más a instancias grandes.